

React Hooks: A Comprehensive Overview



Piyush Kashyap · [Subscribe](#)

4 min read · Dec 13, 2024



Listen



Share

React has revolutionized front-end development by introducing hooks, a powerful feature that lets developers manage state, handle side effects, and optimize performance in functional components. But with so many hooks available, figuring out which ones to use can be overwhelming. In this guide, we'll explore all the React hooks, categorize them into meaningful groups, and provide actionable insights into when and how to use each one.

Whether you're a beginner or a seasoned developer, this article will serve as your go-to resource for mastering React hooks. Let's dive in!

Table of Contents

1. What Are React Hooks?
2. The 8 Major Categories of React Hooks
3. State Management Hooks

- `useState`
- `useReducer`
- `useSyncExternalStore`

4. Effect Hooks

- `useEffect`
- `useLayoutEffect`

- `useInsertionEffect`

5. Ref Hooks

- `useRef`
- `useImperativeHandle`

6. Performance Hooks

- `useMemo`
- `useCallback`

7. Context Hook

- `useContext`

8. Transition Hooks

- `useTransition`
- `useDeferredValue`

9. Other Random Hooks

- `useDebugValue`
- `useId`

10. Conclusion and Best Practices

What Are React Hooks?

React hooks are special functions that let you “hook into” React features. Introduced in React 16.8, they enable functional components to manage state and side effects, replacing the need for class components in most scenarios.

Why Hooks Matter:

- **Cleaner Code:** Eliminate the boilerplate of class components.
- **Reusable Logic:** Custom hooks allow logic sharing between components.

- **Improved Readability:** Hooks make it easy to understand the lifecycle and state management of components.

Now, let's break down React hooks into categories to understand their purposes better.

The 8 Major Categories of React Hooks

React hooks can be grouped into the following categories:

1. **State Management Hooks:** Manage state within components.
2. **Effect Hooks:** Handle side effects and lifecycle events.
3. **Ref Hooks:** Access and manipulate DOM elements or mutable values.
4. **Performance Hooks:** Optimize rendering and computations.
5. **Context Hook:** Access context values in nested components.
6. **Transition Hooks:** Improve user experience with non-urgent updates.
7. **Random Hooks:** Utility hooks for specific use cases.
8. **New React 19 Hooks:** Advanced hooks introduced in React 19.

Let's examine each category in detail.

1. State Management Hooks

`useState` : The Bread and Butter of React Developers

`useState` is the most commonly used hook. It helps manage state within functional components.

How It Works:

- Takes an initial value as an argument.
- Returns an array with two elements:
 1. The current state.
 2. A function to update the state.

Use Cases:

- Managing form inputs like text fields and dropdowns.
- Toggling visibility of modals, tooltips, or components.
- Handling counters or numeric calculations.

```
const [count, setCount] = useState(0);
```

```
return (  
  <button onClick={() => setCount(count + 1)}>  
    Increment: {count}  
  </button>  
);
```

`useReducer` : Simplify Complex State Management

`useReducer` is ideal for managing complex state logic or multiple related state variables.

How It Works:

- Takes a reducer function and initial state as arguments.
- Returns the current state and a `dispatch` function.

Use Cases:

- Managing forms with multiple inputs.
- State logic dependent on previous values (e.g., game state).

```
const reducer = (state, action) => {  
  switch (action.type) {  
    case 'increment':  
      return { count: state.count + 1 };  
    case 'decrement':  
      return { count: state.count - 1 };  
    default:  
      throw new Error();  
  }  
}
```

```
}  
};
```

```
const [state, dispatch] = useReducer(reducer, { count: 0 });
```

`useSyncExternalStore` : **Rarely Used**

`useSyncExternalStore` integrates non-React state stores with React components. Rarely needed unless creating a custom state management library.

2. Effect Hooks

`useEffect` : **Manage Side Effects**

`useEffect` synchronizes components with external systems like APIs or browser DOM.

How It Works:

- Takes a function to run after render.
- Optional dependencies array controls when the effect runs.

Open in app ↗

Sign up

Sign in

Medium

🔍 Search



- Ideal for syncing React state with browser APIs.

```
useEffect(() => {  
  document.title = `Count: ${count}`;  
}, [count]);
```

`useLayoutEffect` : **For Synchronous Side Effects**

Runs before React paints the UI, making it useful for layout-related state updates like measuring DOM elements.

```
useLayoutEffect(() => {  
  const height = ref.current.getBoundingClientRect().height;  
  setHeight(height);  
}, []);
```

`useInsertionEffect` : **Niche Use Case**

Made for CSS-in-JS libraries, `useInsertionEffect` ensures styles are inserted before layout calculations. Most developers won't need this hook.

3. Ref Hooks

`useRef` : **Mutable Values Without Rerenders**

`useRef` stores values without causing rerenders. It's also used to access DOM elements.

```
const inputRef = useRef(null);
```

```
return (  
  <input ref={inputRef} />  
);
```

`useImperativeHandle` : **Expose Custom Methods**

Used alongside `forwardRef` to expose methods to parent components.

```
useImperativeHandle(ref, () => ({  
  focus: () => inputRef.current.focus()  
}));
```

4. Performance Hooks

`useMemo` : **Cache Expensive Computations**

Memoizes a value, recomputing only when dependencies change.

```
const sum = useMemo(() => calculateSum(numbers), [numbers]);
```

`useCallback` : **Memoize Functions**

Prevents functions from being recreated unnecessarily, improving performance in child components.

```
const increment = useCallback(() => setCount(c => c + 1), []);
```

5. Context Hook

`useContext` : **Access Context Values**

Simplifies consuming context values passed down from a provider.

```
const theme = useContext(ThemeContext);
```

6. Transition Hooks

`useTransition` : **Manage Non-Urgent Updates**

Defers state updates, keeping the app responsive.

```
const [isPending, startTransition] = useTransition();
```

```
startTransition(() => setFilter(value));
```

`useDeferredValue` : **Defer Updates Automatically**

Automatically schedules updates for optimal performance.

```
const deferredValue = useDeferredValue(value);
```

7. Other Random Hooks

`useDebugValue` : Debugging Custom Hooks

Labels custom hooks for easier identification in React DevTools.

`useId` : Generate Unique IDs

Generates unique IDs for form elements.

```
const id = useId();
```

Conclusion and Best Practices

React hooks simplify component development, but knowing when and how to use them is key. Here are some best practices:

- **Start Simple:** Use `useState` and `useEffect` as your primary hooks.
- **Optimize:** Use performance hooks like `useMemo` only when necessary.
- **Organize:** Group related state updates with `useReducer`.
- **Experiment:** Try transition hooks for smoother user experiences.

React hooks are a game-changer, making your code cleaner, more reusable, and easier to maintain. With this guide, you're well-equipped to use them effectively. Happy coding!

React

Hooks



Subscribe



Written by Piyush Kashyap

245 Followers · 4 Following

AI/ML Developer turning boring tasks into automated magic with algorithms (no caffeine needed). Models that think for you, with a dash of sarcasm for flavor.

No responses yet



Write a response

What are your thoughts?

More from Piyush Kashyap





Piyush Kashyap

Comprehensive Guide: Installing Docker and Docker Compose on Windows, Linux, and macOS

Docker and Docker Compose have become essential tools for developers and system administrators, enabling the creation and management of...

Dec 7, 2024  8



Remote Desktop Connection

Computer:



User name: None specified

You will be asked for credentials when you connect.



Piyush Kashyap

How to Install XRDP on an Ubuntu Machine: A Simple Guide

XRDP is an open-source implementation of the Microsoft Remote Desktop Protocol (RDP) that allows you to control a remote system...

Jul 18, 2024  5  1



Adding Memory Retention

Integrate memory retention features to enhance chatbot interactions.

Implementing Basic Chatbot

Develop the foundational chatbot structure and functionality



Piyush Kashyap

LangGraph: A Step-by-Step Guide to Building an End-to-End Chatbot with Memory Support

In today's world of advanced AI systems, building a chatbot with capabilities to handle dynamic conversations, integrate external tools...

Nov 26, 2024



100



1



Piyush Kashyap

Your Ultimate Roadmap to Becoming an AI Engineer in 2025

Introduction: Riding the AI Wave

See all from Piyush Kashyap

Recommended from Medium



 TheFrontendFeed

Building Javascript Promises From Scratch: A Frontend Interview Guide

This article focuses on implementing a Promise from scratch—a common interview question for mid to senior frontend positions.

★ Jan 4 🖱️ 74

REACT 19

New features and Updates



In Front-end World by Komal Raut

React 19 Memoization: No More useMemo & useCallback?

As React developers, we have used useMemo and useCallback for years to keep our applications efficient and avoid uncalled re-renders. It is...



Feb 25



107



1



Vitaliy Korzhenko

The Most Difficult JavaScript Interview Question

Can You Answer It ?



 In Dev Simplified by Neha Gupta

Why Companies Are Saying GoodBye to Next.js?

Are you using Next.js or planning to for your next project? Then you need to know this before making a decision!

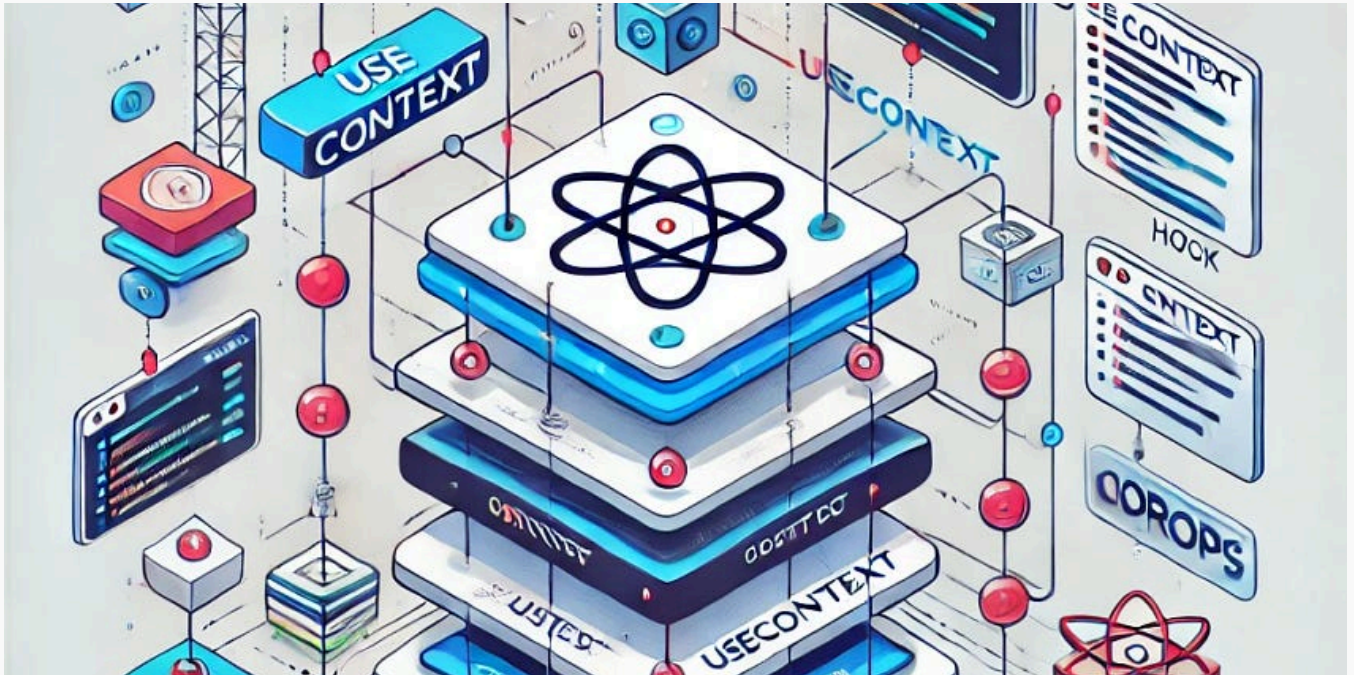


 Abhi Kshirsagar

Redux vs Redux Toolkit: Simplifying State Management in Modern Applications

State management is a cornerstone of modern web application development, especially for applications with complex interactions and data...

★ Dec 10, 2024 🖱 66



 CodeByUmar

Understanding the useContext Hook in React

★ Dec 12, 2024 🖱 52 💬 1



See more recommendations